Ders 1 Çalışma Özeti

Ders 1 Çalışma Özeti

Genel Konular

- Dersin tanıtımı: kaynak kitaplar, içerik, değerlendirme ve proje.
- Yazılım nedir? Yazılımın elemanları ve "yazılım = program" yanılgısı.
- Yazılım yaşam döngüsü (software life cycle) ve bakım (maintenance).
- Yazılım türleri: sistem, mühendislik/bilimsel, gömülü, uygulama, hazır paket (COTS), ürün hattı (product line), kurumsal yazılımlar.
- Eski yazılım (legacy software) kavramı.
- Müşteri, yönetim ve programcı açısından yazılım hakkındaki yanılgılar (software myths).
- Mühendislik kavramı: ölçme ve tasarım.
- Yazılım mühendisliğinin tanımı (Bauer ve IEEE tanımları).
- CASE araçları (Computer-Aided Software Engineering).
- Yazılım geliştirme sürecinin temel adımları ve destek (şemsiye) aktiviteleri.
- Yazılım konfigürasyon yönetimine giriş.

Hocanın Özellikle Vurguladığı Kısımlar

- **Yazılım eşittir program değildir.** Bu derste bu yanlışı kesinlikle yapmamanız gerekiyor. Yazılım; program, belgeler, veri tanımları/veri tabanı ve test verilerinin tümüdür.
- **Mühendisliğin iki anahtar kelimesi:** ölçme ve tasarım. "Mühendis" kelimesi "hendese" (ölçme bilimi) kökünden gelir. Mühendis ölçen, biçen kişidir.
- **Hemen kod yazmaya başlanmaz.** Önce analiz, sonra tasarım, sonra gerçekleme yapılır.
- **Analiz "ne?", tasarım "nasıl?" sorusuna cevap verir.**
  - Analiz: Müşteri ne istiyor? İş süreci nasıl işliyor? Hangi raporlar, hangi kalıcı veriler gerekiyor?
  - Tasarım: Bu iş bilgisayar ortamına nasıl aktarılacak? Yazılım hangi modüllerden oluşacak?
- **Ekonomik, ucuz demek değildir.** Ekonomik çözüm, maliyete göre en fazla faydayı sağlayan çözümdür. En düşük maliyetli çözüm her zaman en ekonomik değildir.
- **Kalite her aşamada ölçülür.** "Yazılımı bitirmeden kalitesini ölçemem" bir yanılgıdır. Örneğin gereksinim analizinin çıktısı olan SRS raporunun doğruluğu da ölçülür.
- **Teslimden sonra iş bitmez.** Yazılıma harcanan emeğin yaklaşık bir o kadarı teslimden sonra harcanır. Hoca, eski bir hocasının "arada kullanıcılara çay kahve içmeye gitmek gerekir" sözünü aktardı.
- **Gereksinimleri sonradan değiştirmenin bedeli üstel olarak artar.**
- Hoca bu derste nesneye yönelik analiz ve tasarımın UML diyagramlarıyla projede uygulanmasını istiyor.

Kısa Tekrar Notları

- **Yazılım:** Herhangi bir donanımda çalışan program + basılı/elektronik belgeler + veri tanımları/veri tabanı + test verileri.
- **Yazılım eskimez ama yetersiz kalır.** Bakım nedenleri:
  - Hata düzeltme.
  - Programlama dili, veri tabanı, işletim sistemi gibi ortamdaki yenilikler.
  - Müşterinin iş sürecindeki değişiklikler.

- **Yazılım yaşam döngüsü:** Fikrin doğmasından geliştirmeye, bakımlı kullanıma, desteksiz kullanıma ve kullanım dışı bırakılmaya kadar geçen süreç.
- **COTS (Commercial Off-The-Shelf):** Satışa hazır paket yazılım. Örnek: hastane yazılım paketi.
- **Ürün hattı (product line):** Bir yazılım evinin bir alanda ortak altyapı, alan analizi (domain analysis) ve alana özgü dille sürekli geliştirdiği ürün ailesi.
- **Yeniden kullanılabilirlik (reusability):** Bir çalışmanın başka işlerde tekrar emek harcamadan kullanılabilmesi. Ürün hatları, COTS ve tasarım kalıpları bunu destekler.
- **Eski yazılım (legacy software):** Uzun süredir kullanılan, iş sürecinin kritik parçası olan yazılım.
  - Eksik/hatalı belgeleri vardır.
  - Yamalarla karmaşıklaşmış, esnek olmayan bir kodu vardır.
  - Örnek: COBOL ile yazılmış bordro ve muhasebe yazılımları.
- **Yazılım mühendisliğinin üç anahtar kelimesi:** kaliteli yazılım, ekonomik yol, mühendislik prensipleri.
- **IEEE tanımı:** Yazılımın geliştirilmesine, işletilmesine ve bakımına sistematik, disiplinli ve ölçülebilir bir yaklaşımın uygulanması.
- **Temel adımlar:** gereksinim analizi, tasarım, gerçekleme (kodlama + veri tabanı), test.
- **Destek aktiviteleri:** proje planlama, kalite yönetimi, yazılım ölçümü, risk analizi, yazılım konfigürasyon yönetimi.
- **Yazılım konfigürasyonu:** Yazılımı oluşturan tüm elemanlar ve aralarındaki ilişkiler. Bunların yönetimi yazılım konfigürasyon yönetimidir.

Detaylı Açıklamalar

Dersin İşleyişi

- **Kaynak kitaplar:**
  - Roger S. Pressman, *Software Engineering: A Practitioner's Approach*, McGraw-Hill, 2014, 8. baskı. Dersin temel kitabı.
  - Ian Sommerville, *Software Engineering*, Pearson, 2015, 10. baskı. Bir grup hoca Türkçeye çevirmiş.
  - Craig Larman, *Applying UML and Patterns: An Introduction to OOAD and Iterative Development*, Prentice-Hall, 2004, 3. baskı. Nesneye yönelik analiz, tasarım ve UML için.
- **İçerik:** Yazılım mühendisliğine giriş, süreç modelleri, gereksinim mühendisliği, nesneye yönelik çözümleme ve tasarım, test teknikleri, proje yönetimine giriş, yazılım ölçütleri (software metrics), konfigürasyon ve risk yönetimi.
- **Değerlendirme:** Dönem içi %60 (vize ve proje arasında paylaştırılacak), final %40.
- **Proje:** Ekip çalışması, yaklaşık 5 kişilik gruplar. Konuları hocalar verecek. Farklı gruplardan karma ekip olmayacak.
- Sınavın test mi klasik mi olacağına daha sonra karar verilecek.
- Ders Sistem Analizi ve Tasarımı dersinin devamı gibi. Orada "bilgi sistemi geliştirme süreci" görülmüştü. Bu derste yalnızca yazılım geliştirme adımları, daha ayrıntılı ve nesneye yönelik kavramlarla işlenecek.
- Bilimsel hazırlık ve mühendislik tamamlama öğrencilerinin önceki derslerin kaynaklarına (UML, nesneye yönelik programlama) ayrıca bakması gerekiyor.
- **Yazılım mühendisliği kitaplarının dört ana kısmı:**
  1. Yazılım mühendisliğine giriş ve süreç modelleri.
  2. Yazılım mimarisi (software architecture). Lisansüstünde ayrı bir ders olarak var.
  3. Yazılım proje yönetimi: zaman ve kaynak planı, risk yönetimi.
  4. Kalite yönetimi ve test teknikleri. Bölümde lisans seçmeli dersi olarak da var.

Yazılım ve Yazılım Yaşam Döngüsü

- Yazılım fiziksel değil, "soft" bir üründür. Makine parçası gibi aşınmaz.
- Ama zamanla yetersiz kalır. Örneğin yeni bir işletim sistemi ya da veri tabanı sürümü altında çalışamaz hale gelebilir. Değişim kaçınılmazdır.
- Bu değişikliklere **bakım (maintenance)** denir.

- Bir süre sonra ne kadar bakım yapılırsa yapılsın yazılım kullanılamaz hale gelir. Kullanımdan kaldırılıp yenilenmesi gerekir.
- Bilgisayar mühendisi yaşam döngüsünün her aşamasında görev alabilir:
  - Proje fikrini oluşturan ekip.
  - Fizibilite ve gereksinim analizi.
  - Yazılım mimarı (software architect) olarak tasarım.
  - Kodlama ekibi.
  - Test ekibi (tester).
  - Bakım ekibi.

Yazılım Türleri

Tür	Örnek
Sistem yazılımları	İşletim sistemleri, derleyiciler
Mühendislik ve bilimsel yazılımlar	Büyük veri, veri madenciliği, istatistiksel analiz
Gömülü (embedded) yazılımlar	Donanım denetimi, gerçek zamanlı uygulamalar
Uygulama yazılımları	Muhasebe, otomasyon, Word, Excel
Hazır paketler (COTS)	Hastane yazılım paketi
Ürün hatları (product line)	Bir yazılım evinin aynı altyapıyla geliştirdiği ürün ailesi
Kurumsal yazılımlar	ERP paketleri, kuruma özel iş yazılımları

- Ürün hattı, fabrikalardaki üretim hattı fikrinin yazılıma uyarlanmasıdır.
- Kurumsal yazılımlar ya müşteriye özel tasarlanır ya da hazır paketler (ör. ERP) kuruma uyarlanır.

Eski Yazılım (Legacy Software)

- Bir şirketin iş sürecinin önemli bir parçasıdır ve çok uzun süredir kullanılır. Örnek: muhasebe, stok kontrol, bordro yazılımları.
- Herkes dokunmaya korkar.
- Özellikleri:
  - Eksik veya hatalı belgeleme.
  - Eklemeler ve yamalarla karmaşıklaşmış kod.
  - Esnek olmayan yapı.
  - Donanımla daha sıkı ilişki.
  - Yazılım mühendisliğindeki gelişmelerden yoksun, daha düşük kalite.
- Yeni gereksinimler ve güncel sistemlerle uyum için değiştirilmek istenir. Ama bakımı zor olduğu için bir süre daha yamalarla kullanılır.
- Örnek: 2000 yılına geçişte (Y2K) Avrupa'da COBOL programcıları arandı.

Yazılım Hakkındaki Yanılgılar

Bir yazılım projesinin üç tarafı vardır: müşteri, yönetim ve yazılım geliştirici.

- **Müşteri tarafı:**
  - "Başlamak için amaçları genel olarak belirtmek yeter, ayrıntılar sonra." Yanlış. Gereksinimler başta belirtilmezse temel doğru atılmaz.
  - "Yazılım esnektir, değişen gereksinimler kolayca eklenir." Yanlış. Yaşam döngüsü ilerledikçe değişikliğin bedeli üstel olarak artar.
  - Gereksinim, işin nasıl yapıldığının anlatılmasıdır. Önce iş süreci öğrenilir.
- **Yönetim tarafı:**
  - "İş yetişmiyorsa ekibe yeni programcı ekleriz." Yanlış. Geciken projeye sonradan eleman eklemek işi daha da geciktirir. (Bu, literatürde Brooks yasası olarak bilinir.)
  - "İşin tamamını ya da bir kısmını fason yaptırırım." Dış kaynak kullanımı ancak baştan planlanırsa doğrudur. Örneğin ekran tasarımları bir grafik tasarım firmasına verilebilir.

  - Kendi içindeki ilerlemeyi denetleyemeyen firma, dışarıya verdiği işi de izleyemez.
  - "Açık kaynak yazılım üretirsem kâr edemem." Kâr danışmanlık hizmetlerinden ve farklı iş modellerinden gelir.
- **Geliştirici tarafı:**
  - "Teslim edince işim biter." Yanlış. Emeğin yaklaşık yarısı teslimden sonra harcanır. Kurulum, veri arttığında oluşan sorunlar ve kullanıcı alışması takip edilmelidir.
  - "Bitirmeden kaliteyi ölçemem." Yanlış. Her aşamanın kalite ölçütleri vardır.
  - "Yazılım = program." Yanlış. SRS raporu, tasarım raporu, test, entegrasyon, bakım ve kullanıcı belgeleri de yazılımın parçasıdır.
  - "Önemli olan çalışan programdır, ben iyi algoritma yazarım." Yanlış. İşin belgelenmesi ve başkalarının anlayacağı hale getirilmesi gerekir.
- Bu kadar çabaya rağmen yazılım projelerinin önemli bir kısmı hâlâ başarısız oluyor. Hoca bunu proje yönetiminin önemine bağladı.

Mühendislik ve Yazılım Mühendisliği

- Derste bir öğrencinin tanımı: "Bir sorunla karşılaştığında çözümünü sağlayacak sistemi geliştirebilen kişi."
- Hoca ekledi: İlk akla gelen çözüm hemen uygulanmaz. Önce **tasarım** yapılır.
- Türkiye'de mühendislik programlarını MÜDEK, uluslararası alanda ABET akredite eder. İkisi de programlarda tasarım çalışmalarına bakar.
- **Ölçme:** Geliştirdiğimiz ya da geliştireceğimiz yazılımın büyüklüğünü ve karmaşıklığını ölçebiliyor muyuz?
- **Yazılım ölçütleri (software metrics):** Bir yazılımı neye göre ölçeceğiz?
  - İç ve dış özellikler.
  - Maliyet ve satış fiyatı belirleme.
  - Proje başında maliyet tahmini.
  - Büyüklük, karmaşıklık ve algoritma yapısı ölçütleri.
- **Klasik tanım:** Kaliteli yazılımı en ekonomik yolla elde etmek üzere mühendislik prensiplerinin kullanılması.
  - Kaliteli yazılım: Bir süreç modeli benimseyip standartlarıyla geliştirmek. En iyi yaklaşımı ve algoritmayı seçmek, bakım yapacakların anlayacağı şekilde belgelemek.
  - Ekonomik yol: Proje yönetimi, zaman ve kaynak planlaması.
  - Mühendislik prensipleri: Ölçme ve tasarım.
- **IEEE tanımı:** Yazılımın geliştirilmesine, işletilmesine ve bakımına, yani yaşam döngüsüne, sistematik, disiplinli ve ölçülebilir bir yaklaşımın uygulanması.

CASE Araçları

- **CASE (Computer-Aided Software Engineering):** CAD/CAM'in yazılım geliştirmedeki karşılığı. Süreçteki işleri kolaylaştıran, hızlandıran ve rutin işleri yapan yazılımlar.
- Örnekler:
  - UML diyagramlarını standartlara uygun çizmeye yardım eden araçlar.
  - Use case diyagramıyla uyumlu sıralama (ardışıl) diyagramı çizmeyi destekleyen, analiz ve tasarım arasında tutarlılık kontrolü yapan araçlar.
  - ER diyagramı ile veri akış diyagramı arasında tutarlılık sağlayan araçlar.
- **Violet:** Nesneye yönelik modelleme için basit bir UML aracı. Projelerde kullanılması istenecek.
- **Bütünleşik CASE araçları:** Yazılım geliştirmenin her aşamasını birbiriyle uyumlu destekleyen platformlar. Örnek: Rational Rose / Rational ailesi. Şu anda IBM'e ait.

Yazılım Geliştirme Sürecinin Adımları

Temel adımlar:

1. **Gereksinim analizi (requirements analysis):**
   - Müşterinin ne istediğini anlamak.
   - İş analizi, raporlar, saklanacak veriler.
   - Use case ve veri akış diyagramları.
   - Anlaşılanı müşteriye doğrulatmak.

    - Bu derste sistem analizindeki 3 adım (bilgi toplama, modelleme, raporlama) yerine 6-7 aşama görülecek.
    - Bazı İTÜ hocaları "ister çözümleme" terimini kullanıyor. Bu derste "gereksinim analizi" kullanılıyor.
2. **Tasarım (design):**
    - İşin bilgisayar ortamına nasıl aktarılacağı.
    - Modüllere ayırma.
    - Klasik yaklaşımda yapı diyagramı (structure chart), nesneye yönelik yaklaşımda tasarım modelleri.
3. **Gerçekleme (implementation):** Yalnızca kod değil, veri tabanının oluşturulması da bu adımdadır. Bu yüzden "kodlama" yerine "gerçekleme" demek daha doğru.
4. **Test.**

- Hoca, sistem analizi dersindeki "salıncak hikâyesi" karikatürünü hatırlattı. Müşteri basit bir salıncak ister. Analist, tasarımcı ve programcı her biri başka bir şey anlar. Sonuçta istenmeyen bir ürün çıkar. Belgeleme yoksa durum daha da kötüleşir.

Destek (şemsiye) aktiviteleri:

- **Proje planlama:** Kaynak planı, zaman planı, görev atama, projenin takibi.
- **Kalite yönetimi:** Kalite planı, kalite ekibi, kullanılacak standart.
- **Yazılım ölçümü:** Hazırlanan yazılımın ölçülmesi, başta istenen büyüklüğün belirlenmesi.
- **Risk analizi:** Geliştirme sürecindeki risklerin belirlenmesi ve sürekli takibi.
- **Yazılım konfigürasyon yönetimi:** Yazılımın tüm elemanları ve aralarındaki ilişkilerin yönetimi.

Sonraki Hafta

- Süreç adımları netleştirilecek.
- Yazılım geliştirme süreç modelleri: klasik modeller ve çevik (agile) modeller.

Ders sonrası notlar

- **"Yazılım mühendisliği kavramını 70'lerde Boehm ortaya attı" ifadesi düzeltilmeli.** "Software engineering" terimi 1968'deki NATO Yazılım Mühendisliği Konferansı ile yaygınlaştı. Barry Boehm'in 1976 tarihli "Software Engineering" makalesi ise alanın önemli erken çalışmalarından biridir.
- **"Kaliteli yazılımı ekonomik yolla..." tanımı** Fritz Bauer'e aittir (1968–1969). Özgün hali: güvenilir ve gerçek makinelerde verimli çalışan yazılımı ekonomik olarak elde etmek için sağlam mühendislik ilkelerinin kurulması ve kullanılması.
- **IEEE tanımı** IEEE Std 610.12-1990 sözlüğündendir. Pressman bunu 1993 tarihli IEEE standartlar derlemesinden aktarır. Hocanın "1993" demesi bu kaynaktan geliyor.
- **MÜDEK'in açılımı** "Mühendislik Eğitim Programları Değerlendirme ve Akreditasyon Derneği"dir. Hocanın söylediği "mühendislik fakültesi dekanları konseyi" (MDK) farklı bir kuruldur. MÜDEK bu konseyin çalışmalarından doğmuştur.
- **Altyazıdaki bozuk yerler:** "Code şu commercial order şerf" = COTS (Commercial Off-The-Shelf). "IR diagramı" = ER (Entity-Relationship) diyagramı. "Software Matrix" = software metrics. "Jizk analizan" = risk analizi. "PEST" = test.
- **Yazılım ve aşınma:** Pressman'a göre yazılım aşınmaz (wear out) ama bozulur (deteriorate). Her değişiklik yeni hatalar getirebilir ve yazılımın hata oranı zamanla yükselir.

Not: İsterseniz bu dersin altyazı (.srt) dosyasını Gemini Notebook gibi bir yapay zeka aracına yükleyerek ders hakkında daha detaylı soru-cevaplar yapabilir ve dersi verimli çalışabilirsiniz.